School of Computer Science Engineering and Information Systems
BCSE302L – Database Systems | Course Project

EMPLOYEE PAYROLL MANAGEMENT SYSTEM

Review III – Integrated System & Final Demonstration

Submitted by

Krishiv Rajkumar – 24BDE0132
Asher Mathew – 24BDE0128
Sai Praneeth – 24BYB0143

Under the guidance of
Dr Rubimalaya R

October 2026

Project link: https://dbs-proj-mauve.vercel.app

Contents

Contents 2
1. Project Title and Team Details 4
2. Introduction and Problem Statement 5
2.1 Introduction 5
2.2 Problem Statement 5
2.3 Objectives 5
2.4 Users and Roles 5
3. Functional Requirements 7
4. ER Diagram 8
5. Relational Schema 9
5.1 Relational Schema (text notation) 10
6. Table Creation and Constraints 11
6.1 Constraint Summary 11
6.2 Data Population 11
6.3 DML Demonstration 12
7. SQL Queries and Outputs 13
Q4. Department-wise salary cost for the latest payroll month 13
Q5. Top 5 earners by net salary in the latest month (with rank) 14
Q8. Remaining 2026 leave balance of every active employee 14
Q14. Departments whose average net salary is above the company average (latest month) 15
8. Procedures, Functions, Cursor and Triggers 16
8.1 Overview 16
8.2 Procedure with Cursor – generate_monthly_payroll 16
8.3 Trigger – trg_leave_approved 18
8.4 Calling Procedures from the Application 18
9. Real-World Transaction Demonstrations 20
9.1 Application and Database Connectivity 20
9.2 Integrated Workflow – One Monthly Payroll Cycle 20
9.3 Transactions 21
T1. Employee checks in 21
T2. Employee applies for leave 22
T3. Manager approves leave 22
T4. Payroll officer generates the monthly payroll 23
T5. Viewing an itemised payslip 23
T6. Paying the month’s salaries 24
T7. Attempt to delete a paid payslip 24
T8. Employee resignation 25

T9. Running the report queries from the application 25
9.4 Reports Generated from the Database 26
9.5 Validation – Invalid Operations Rejected by the Database 27
10. Conclusion 28
10.1 Design Decisions and Justification 28
10.2 Conclusion and Future Work 28

(If page numbers are missing, right-click the table and choose "Update Field".)

1. Project Title and Team Details

Item	Details
Project title	Employee Payroll Management System
Course	BCSE302L – Database Systems (Lab course project)
Team	Krishiv Rajkumar (24BDE0132), Asher Mathew (24BDE0128), Sai Praneeth (24BYB0143)
Guide	Dr Rubimalaya R
Database	PostgreSQL 17 hosted on Supabase (cloud); PL/pgSQL for procedures, functions, cursors and triggers
Application	React + Vite web application connected to the database through the supabase-js client
Project link	https://dbs-proj-mauve.vercel.app
Source code	github.com/LordFlyaway/DBS_proj
Sample company	All Caps Pvt. Ltd. – a fictional company whose 20 employees are named after musicians and athletes

2. Introduction and Problem Statement

2.1 Introduction

Every organisation has to pay its employees correctly and on time. A salary is not one fixed number: it depends on the designation and basic pay, allowances such as HRA and DA, deductions such as Provident Fund, Professional Tax and TDS, the days actually worked, and approved or unapproved leave. When this is managed in spreadsheets, calculations go wrong, records are duplicated, nothing stops invalid entries and there is no audit trail.

The Employee Payroll Management System stores organisation, employee, attendance, leave and salary-structure data in a relational database and generates monthly payroll and payslips from that data with stored procedures. Business rules are enforced inside the database through constraints, triggers and Row Level Security, and a role-based web application lets HR, payroll staff, managers and employees work with the data.

2.2 Problem Statement

Design and implement a centralised, consistent and secure database that (a) maintains employee and organisational data, (b) records daily attendance and leave, (c) defines salary components (allowances and deductions), and (d) computes each employee's monthly gross and net salary, including loss-of-pay for unpaid absence, while enforcing business rules with database constraints and triggers instead of manual checks.

2.3 Objectives

- A single source of truth for employees, departments and designations.
- Attendance and leave recording, with loss-of-pay calculated automatically.
- Configurable salary components – fixed amounts or a percentage of basic pay.
- One-step monthly payroll generation with itemised payslips, followed by salary payment.
- Integrity through PK/FK, NOT NULL, UNIQUE, CHECK and DEFAULT constraints, triggers and Row Level Security.
- Interactive access through a web application connected to the database.

2.4 Users and Roles

Role (demo login)	**What the role can do**
Admin – Daniel Dumile (HR Manager)	Manage employees, departments, designations and attendance; approve any leave; run payroll; run the SQL query catalogue.
Payroll – Jeffery Williams (Payroll Officer)	Maintain salary components; generate, pay and (if unpaid) delete payroll; run the SQL query catalogue.
Manager – Aubrey Graham (Engineering Manager)	See their team; approve or reject the team's leave; follow team attendance.
Employee – LeBron James (Sr. Software Engineer)	Check in and out; apply for and cancel leave; view own attendance and payslips.

Every role also sees its own dashboard, leave balance and payslips. Logins are e-mail/password accounts in Supabase Auth, linked to EMPLOYEE rows through `employee.auth_user_id` and authorised through `employee.app_role`.

3. Functional Requirements

ID	Requirement	Implemented by
FR1	Add, update and delete departments, including the department head.	DEPARTMENT; Departments page
FR2	Maintain designations with pay grade, basic salary and annual leave quota.	DESIGNATION; Designations page
FR3	Register employees with personal, PAN and bank details, department, designation, manager and login role.	EMPLOYEE; CHECK/UNIQUE constraints; trg_validate_manager
FR4	Record attendance – one row per employee per day; employees check themselves in and out.	ATTENDANCE; UNIQUE (emp_id, att_date); RLS policy
FR5	Employees apply for leave; managers approve or reject; approved leave appears in attendance.	LEAVE_REQUEST; process_leave; trg_leave_approved
FR6	Define allowances and deductions as fixed amounts or a percentage of basic.	SALARY_COMPONENT; fn_component_amount
FR7	Generate the month's payroll for all active employees with LOP, gross and net salary.	generate_monthly_payroll (cursor); fn_working_days; fn_lop_days
FR8	Keep an itemised payslip (amount of every component).	PAYROLL_DETAIL (M:N bridge)
FR9	Pay salaries: mark payroll Paid with date, mode and transaction reference.	pay_monthly_payroll (cursor FOR UPDATE)
FR10	Prevent invalid operations – duplicate payroll, editing paid payroll, self-approval, exceeding leave balance.	Constraints, procedures and triggers
FR11	Handle employee exits – reassign their team, clear department headship, reject pending leave.	trg_employee_exit
FR12	Management reports: department cost, attendance summary, leave balance, top earners and others.	19 SQL queries saved as views
FR13	Role-based web application connected to the database.	React app + supabase-js + Row Level Security

4. ER Diagram

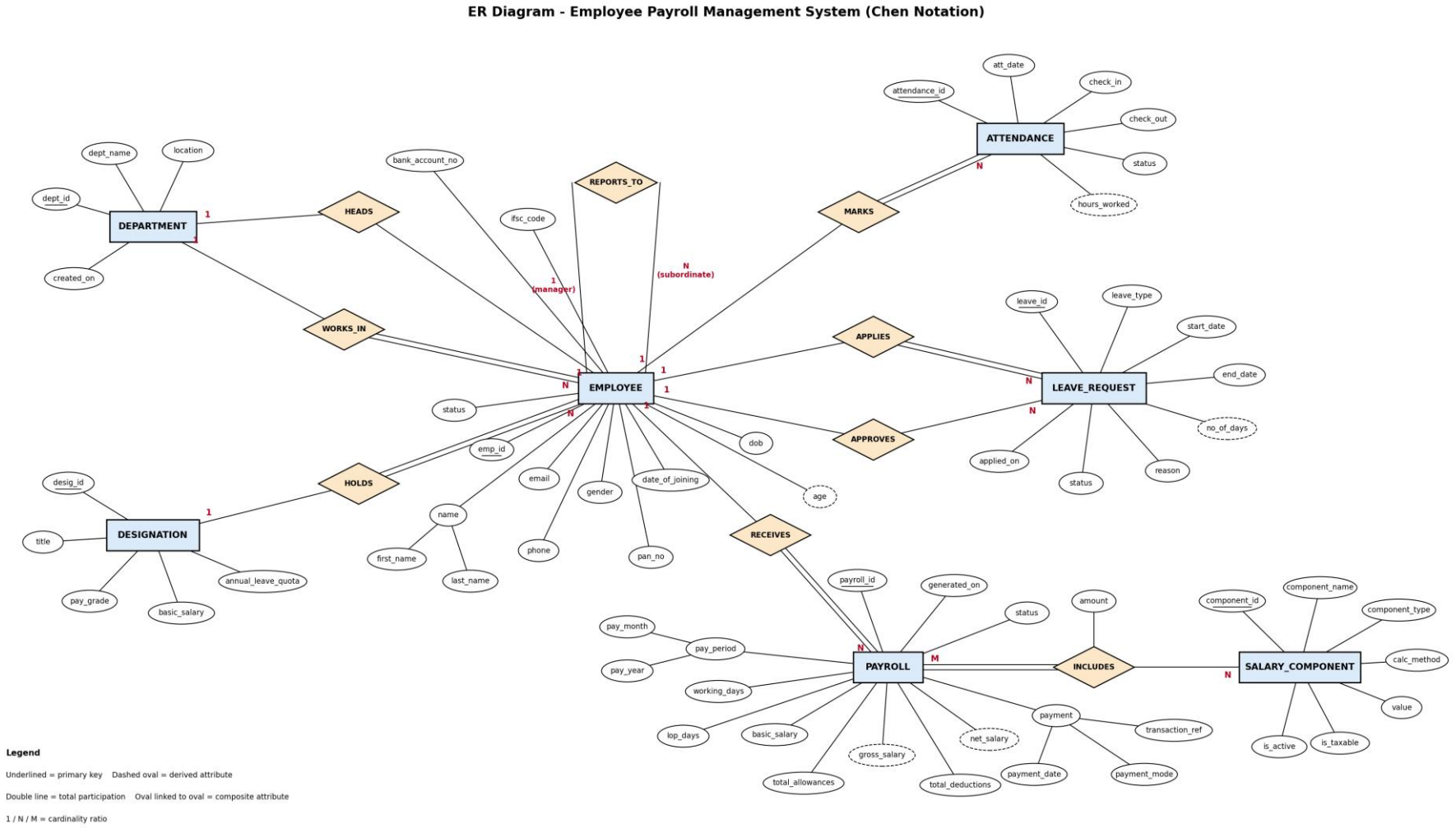


Figure 1 – ER diagram (Chen notation): 7 entities and 9 relationships, including the recursive REPORTS_TO, the 1:1 HEADS and the M:N INCLUDES relationship.

5. Relational Schema

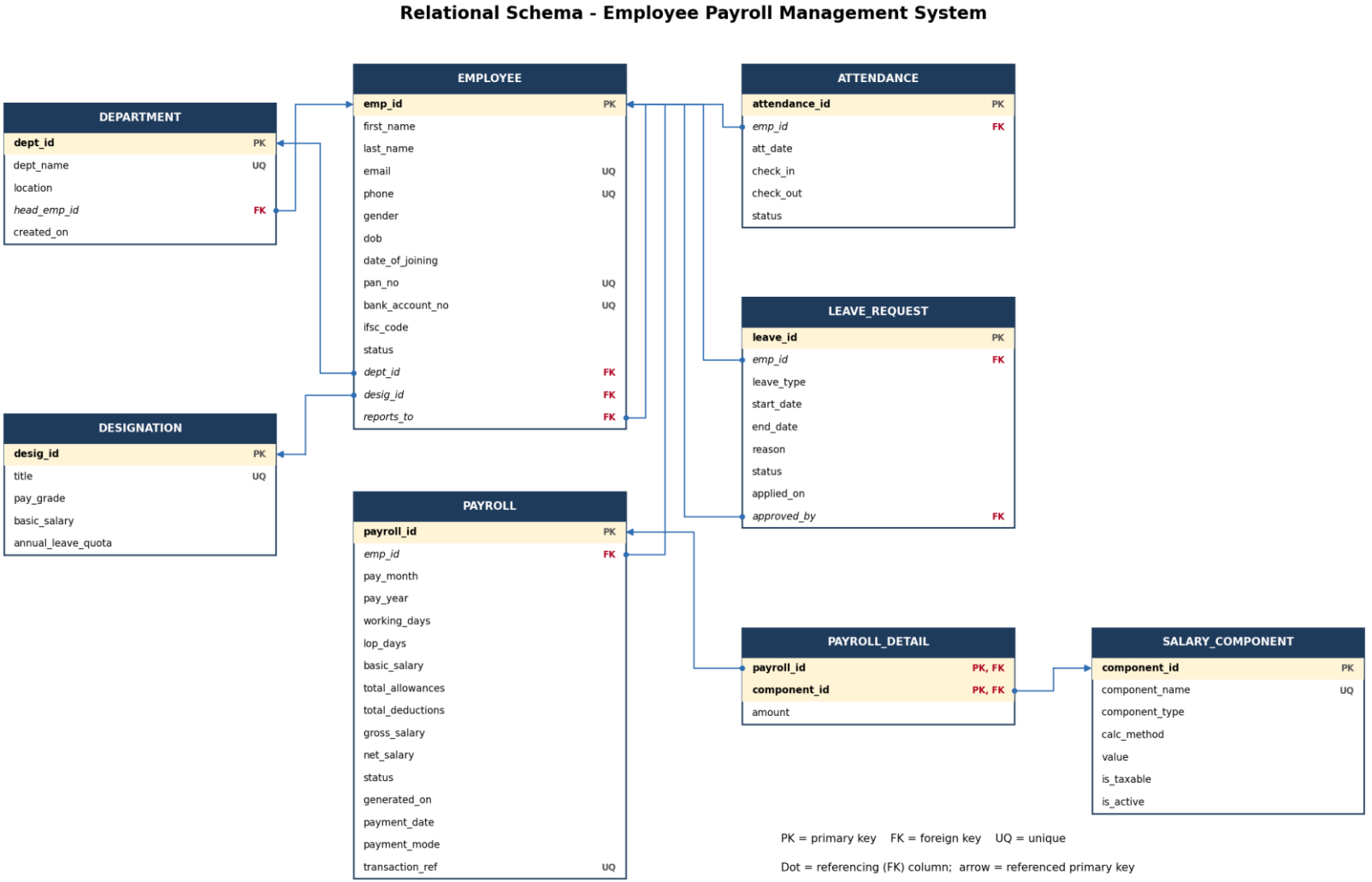


Figure 2 – Relational schema: 8 tables with primary and foreign keys.

5.1 Relational Schema (text notation)

Primary keys are **bold and underlined**; foreign keys are *italic*.

DEPARTMENT (**dept_id**, dept_name, location, *head_emp_id*, created_on)

DESIGNATION (**desig_id**, title, pay_grade, basic_salary, annual_leave_quota)

EMPLOYEE (**emp_id**, first_name, last_name, email, phone, gender, dob, date_of_joining, pan_no, bank_account_no, ifsc_code, status, *dept_id*, *desig_id*, *reports_to*, app_role, auth_user_id)

ATTENDANCE (**attendance_id**, *emp_id*, att_date, check_in, check_out, status)

LEAVE_REQUEST (**leave_id**, *emp_id*, leave_type, start_date, end_date, reason, status, applied_on, *approved_by*)

SALARY_COMPONENT (**component_id**, component_name, component_type, calc_method, value, is_taxable, is_active)

PAYROLL (**payroll_id**, *emp_id*, pay_month, pay_year, working_days, lop_days, basic_salary, total_allowances, total_deductions, gross_salary, net_salary, status, generated_on, payment_date, payment_mode, transaction_ref)

PAYROLL_DETAIL (***payroll_id***, ***component_id***, amount)

Change since Review I: two columns were added to EMPLOYEE for the web application – `app_role` (Admin, Payroll, Manager or Employee) and `auth_user_id` (the employee's Supabase login, UNIQUE). Keeping them in EMPLOYEE keeps the design at 8 tables.

All tables are in Third Normal Form: department and designation details are stored once and referenced by key, salary rules are separate from payslip line items, and the only non-key attribute of the composite-key table PAYROLL_DETAIL (amount) depends on the whole key. Age, hours worked and leave days are derived in queries and are not stored; gross and net salary are stored on purpose as a monthly snapshot so that old payslips do not change when salary rules change.

6. Table Creation and Constraints

The database is created by `sql/01_schema.sql`. Each table and the business rule behind each constraint is summarised below, followed by the complete DDL.

6.1 Constraint Summary

Table	Keys	NOT NULL / UNIQUE / CHECK / DEFAULT
DEPARTMENT	PK dept_id; FK head_emp_id (UNIQUE) → employee	dept_name UNIQUE NOT NULL; created_on DEFAULT CURRENT_DATE
DESIGNATION	PK desig_id	title UNIQUE; pay_grade IN (G1–G5); basic_salary > 0; annual_leave_quota DEFAULT 24, 0–60
EMPLOYEE	PK emp_id; FK dept_id, desig_id, reports_to (self)	email, phone, pan_no, bank_account_no, auth_user_id UNIQUE; phone ^[6-9][0-9]{9}$; PAN ^[A-Z]{5}[0-9]{4}[A-Z]$; IFSC format; gender IN (M,F,O); status DEFAULT 'Active'; app_role DEFAULT 'Employee'; at least 18 at joining; cannot report to self
ATTENDANCE	PK attendance_id; FK emp_id (CASCADE)	UNIQUE (emp_id, att_date); status IN (Present, Absent, Half-Day, Leave) DEFAULT 'Present'; check_out > check_in
LEAVE_REQUEST	PK leave_id; FK emp_id (CASCADE), approved_by (SET NULL)	leave_type IN (Casual, Sick, Earned, Unpaid); end_date ≥ start_date; status DEFAULT 'Pending'; approver ≠ applicant
SALARY_COMPONENT	PK component_id	component_name UNIQUE; type IN (Allowance, Deduction); calc_method IN (Fixed, Percent); value ≥ 0 and ≤ 100 for Percent; is_taxable, is_active DEFAULT TRUE
PAYROLL	PK payroll_id; FK emp_id (RESTRICT)	UNIQUE (emp_id, pay_month, pay_year); month 1–12; lop_days ≤ working_days; net_salary ≥ 0; status DEFAULT 'Generated'; Paid ⇒ payment_date and mode present; transaction_ref UNIQUE
PAYROLL_DETAIL	PK (payroll_id, component_id); FK both (CASCADE / RESTRICT)	amount ≥ 0

The complete DDL is in `sql/01_schema.sql` and was presented in full in the Review II report.

6.2 Data Population

Every table holds at least 10 meaningful rows describing All Caps Pvt. Ltd. from August to October 2026; payroll rows are produced by the payroll procedures, not typed in.

Table	Rows
department	10
designation	13
employee	20
attendance	971
leave_request	13
salary_component	12
payroll	39
payroll_detail	390

6.3 DML Demonstration

INSERT, UPDATE and DELETE statements executed on the populated database, including statements the constraints reject:

#	Operation	Database result
1	INSERT a new employee	Success – 1 row(s)
2	INSERT rejected by CHECK constraint (invalid PAN format)	Rejected – new row for relation "employee" violates check constraint "employee_pan_no_check"
3	INSERT rejected by UNIQUE constraint (attendance already recorded)	Rejected – duplicate key value violates unique constraint "uq_att_emp_day"
4	INSERT rejected by FOREIGN KEY (department does not exist)	Rejected – insert or update on table "employee" violates foreign key constraint "employee_dept_id_fkey"
5	UPDATE: annual salary revision of a designation	Success – 1 row(s)
6	UPDATE: deactivate a salary component	Success – 1 row(s)
7	DELETE: employee cancels a pending leave request	Success – 1 row(s)
8	DELETE: remove the employee added above (no payroll yet)	Success – 1 row(s)
9	DELETE rejected by FOREIGN KEY (employee has payroll history)	Rejected – update or delete on table "employee" violates foreign key constraint "payroll_emp_id_fkey" on table "payroll"

7. SQL Queries and Outputs

Nineteen queries are saved as views in `sql/05_queries.sql` and run live from the SQL Queries page. The table lists all of them; four representative queries are shown in full with their actual output.

#	Question answered	SQL concepts	Rows
Q1	Employee directory with department, designation and manager	INNER JOIN, LEFT SELF-JOIN	20
Q2	Head-count and average basic salary of every department (including empty ones)	LEFT JOIN, GROUP BY, COUNT / AVG	10
Q3	Employees whose basic salary is above the company average	Scalar subquery, Comparison	6
Q4	Department-wise salary cost for the latest payroll month	CTE (WITH), JOIN, SUM, GROUP BY	10
Q5	Top 5 earners by net salary in the latest month (with rank)	Window function RANK(), ORDER BY, LIMIT	5
Q6	Attendance summary of every employee for September 2026	Conditional aggregation (FILTER), GROUP BY	19
Q7	Employees who lost pay (LOP) in any month, with the amount lost	WHERE filter, Arithmetic expression, JOIN	32
Q8	Remaining 2026 leave balance of every active employee	User-defined FUNCTION in SELECT	19
Q9	Pending leave requests and the manager who must approve them	Self-join, Date arithmetic	4
Q10	Managers with at least 3 direct reports	Self-join, GROUP BY, HAVING	3
Q11	Itemised payslip lines of every employee for the latest month	4-table JOIN, M:N bridge table	190
Q12	Active employees who have not taken any approved leave in 2026	NOT EXISTS (correlated subquery)	11
Q13	Month-by-month payroll totals and payment status	GROUP BY multiple columns, SUM, CASE	2
Q14	Departments whose average net salary is above the company average (latest month)	HAVING with subquery, AVG	4
Q15	Age and tenure of every active employee (derived attributes)	Date functions AGE() / EXTRACT, Derived attributes	19
Q16	Total paid out per salary component in the latest month	JOIN, GROUP BY, ROLLUP-style totals with UNION ALL	12
Q17	Employees who were both absent and on approved leave in September	INTERSECT (set operation)	2
Q18	Everyone with leadership responsibility: department heads or people managers	UNION (set operation), IN subquery	11
Q19	Employees who checked in after 09:35 more than 10 times since August	Time comparison, GROUP BY, HAVING	10

Q4. Department-wise salary cost for the latest payroll month

```
WITH latest AS (
    SELECT pay_month, pay_year
      FROM payroll
     ORDER BY pay_year DESC, pay_month DESC
     LIMIT 1
)
SELECT d.dept_name,
       COUNT(*)              AS employees_paid,
       SUM(p.gross_salary)   AS total_gross,
       SUM(p.total_deductions) AS total_deductions,
       SUM(p.net_salary)     AS total_net
  FROM payroll p
  JOIN latest l     ON l.pay_month = p.pay_month AND l.pay_year = p.pay_year
  JOIN employee e   ON e.emp_id = p.emp_id
  JOIN department d ON d.dept_id = e.dept_id
 GROUP BY d.dept_name
 ORDER BY total_net DESC;
```

Output – 10 row(s):

dept_name	employees_paid	total_gross	total_deductions	total_net
Engineering	5	473034.11	51119.32	421914.79

dept_name	employees_paid	total_gross	total_deductions	total_net
Administration	1	235804.55	25040.91	210763.64
Sales	3	222518.21	24238.64	198279.57
Finance	3	218026.37	23767.28	194259.09
Human Resources	2	193374.11	20884.32	172489.79
Legal	1	114672.73	12329.54	102343.19
Quality Assurance	1	75093.18	8176.13	66917.05
Marketing	1	71890.00	7840.00	64050.00
Operations	1	49210.00	5460.00	43750.00
Customer Support	1	43024.55	4810.91	38213.64

Q5. Top 5 earners by net salary in the latest month (with rank)

```
WITH latest AS (
    SELECT pay_month, pay_year
      FROM payroll
     ORDER BY pay_year DESC, pay_month DESC
     LIMIT 1
)
SELECT RANK() OVER (ORDER BY p.net_salary DESC) AS rank,
       e.first_name || ' ' || e.last_name AS employee,
       g.title,
       p.net_salary
  FROM payroll p
  JOIN latest l      ON l.pay_month = p.pay_month AND l.pay_year = p.pay_year
  JOIN employee e    ON e.emp_id = p.emp_id
  JOIN designation g ON g.desig_id = e.desig_id
 ORDER BY p.net_salary DESC
 LIMIT 5;
```

Output – 5 row(s):

rank	employee	title	net_salary
1	Sachin Tendulkar	Director	210763.64
2	Aubrey Graham	Engineering Manager	134638.64
3	Daniel Dumile	HR Manager	117996.60
4	Michael Jordan	Sales Manager	113877.28
5	Mary Kom	Legal Counsel	102343.19

Q8. Remaining 2026 leave balance of every active employee

```
SELECT e.emp_id,
       e.first_name || ' ' || e.last_name AS employee,
       g.annual_leave_quota AS quota,
       fn_leave_balance(e.emp_id, 2026) AS balance
  FROM employee e
  JOIN designation g ON g.desig_id = e.desig_id
 WHERE e.status = 'Active'
 ORDER BY balance, e.emp_id;
```

Output – 19 row(s), first 10 shown:

emp_id	employee	quota	balance
12	PV Sindhu	18	13
10	Simone Biles	20	17
14	Naomi Osaka	20	17
7	Cristiano Ronaldo	20	18
9	Lionel Messi	20	18
13	Virat Kohli	18	18

emp_id	employee	quota	balance
15	Usain Bolt	18	18
16	Sania Mirza	18	18
6	LeBron James	22	20
8	Serena Williams	20	20

Q14. Departments whose average net salary is above the company average (latest month)

```
WITH latest AS (
    SELECT pay_month, pay_year
      FROM payroll
     ORDER BY pay_year DESC, pay_month DESC
     LIMIT 1
)
SELECT d.dept_name,
       ROUND(AVG(p.net_salary), 2) AS avg_net_salary
  FROM payroll p
  JOIN latest l     ON l.pay_month = p.pay_month AND l.pay_year = p.pay_year
  JOIN employee e   ON e.emp_id = p.emp_id
  JOIN department d ON d.dept_id = e.dept_id
 GROUP BY d.dept_name
HAVING AVG(p.net_salary) > (SELECT AVG(p2.net_salary)
                            FROM payroll p2
                            JOIN latest l2 ON l2.pay_month = p2.pay_month AND l2.pay_year =
p2.pay_year)
 ORDER BY avg_net_salary DESC;
```

Output – 4 row(s):

dept_name	avg_net_salary
Administration	210763.64
Legal	102343.19
Human Resources	86244.90
Engineering	84382.96

8. Procedures, Functions, Cursor and Triggers

All server-side logic is written in PL/pgSQL, PostgreSQL's procedural language (the equivalent of Oracle PL/SQL). It lives in `sql/02_functions_procedures.sql` and `sql/03_triggers.sql`.

8.1 Overview

Type	Name	Business purpose	Used by
Function	fn_working_days(month, year)	Working days (Mon–Fri) in a month	generate_monthly_payroll
Function	fn_component_amount(component, basic)	Amount of a fixed or % salary component	generate_monthly_payroll
Function	fn_lop_days(emp, month, year)	Loss-of-pay days: absent + ½ half-day + unpaid leave	generate_monthly_payroll, Q7
Function	fn_leave_balance(emp, year)	Remaining paid leave for the year	process_leave, Q8, Leave page
Procedure + cursor	generate_monthly_payroll(month, year)	Creates every active employee's payroll and payslip lines	Payroll Run page
Procedure	process_leave(leave, approver, decision)	Validates and approves/rejects a leave request	Leave approvals page
Procedure + cursor	pay_monthly_payroll(month, year, mode)	Marks the month's payroll Paid with a transaction reference	Payroll Run page
Trigger	trg_leave_approved	Approved leave → 'Leave' rows in ATTENDANCE	leave_request UPDATE
Trigger	trg_lock_paid_payroll	Paid payroll cannot be changed or deleted	payroll UPDATE/DELETE
Trigger	trg_validate_manager	Manager must be active; no reporting cycles	employee INSERT/UPDATE
Trigger	trg_employee_exit	On resignation: reassign team, clear headship, reject pending leave	employee UPDATE

The complete code of every function, procedure and trigger is in `sql/02_functions_procedures.sql` and `sql/03_triggers.sql`; the two components at the heart of the monthly cycle are shown below.

8.2 Procedure with Cursor – generate_monthly_payroll

```
-- ---------------------------------------------------------------------
-- PROCEDURE 1: generate_monthly_payroll(month, year, INOUT generated)
--   Uses an explicit CURSOR over all active employees. For each one:
--     LOP days -> earned basic -> every active salary component
--     -> PAYROLL row + PAYROLL_DETAIL line items.
--   Employees who already have a payroll for that month are skipped,
--   so the procedure can safely be run again.
-- ---------------------------------------------------------------------
CREATE OR REPLACE PROCEDURE generate_monthly_payroll(
    p_month INT,
    p_year INT,
    INOUT p_generated INT DEFAULT 0)
LANGUAGE plpgsql
AS $$
DECLARE
    v_month_end   DATE := (make_date(p_year, p_month, 1) + INTERVAL '1 month' - INTERVAL '1
day')::DATE;
    v_work_days   INT  := fn_working_days(p_month, p_year);

    cur_emp CURSOR FOR
        SELECT e.emp_id, d.basic_salary
          FROM employee e
```

```
          JOIN designation d ON d.desig_id = e.desig_id
         WHERE e.status IN ('Active', 'On Leave')
           AND e.date_of_joining <= v_month_end
         ORDER BY e.emp_id;

    rec           RECORD;
    comp          RECORD;
    v_lop         NUMERIC;
    v_earned      NUMERIC;
    v_amount      NUMERIC;
    v_allow       NUMERIC;
    v_deduct      NUMERIC;
    v_payroll_id  INT;
BEGIN
    p_generated := 0;
    OPEN cur_emp;
    LOOP
        FETCH cur_emp INTO rec;
        EXIT WHEN NOT FOUND;

        CONTINUE WHEN EXISTS (SELECT 1 FROM payroll
                              WHERE emp_id = rec.emp_id
                                AND pay_month = p_month AND pay_year = p_year);

        v_lop    := LEAST(fn_lop_days(rec.emp_id, p_month, p_year), v_work_days);
        v_earned := ROUND(rec.basic_salary - rec.basic_salary / v_work_days * v_lop, 2);
        v_allow  := 0;
        v_deduct := 0;

        INSERT INTO payroll (emp_id, pay_month, pay_year, working_days, lop_days, basic_salary)
        VALUES (rec.emp_id, p_month, p_year, v_work_days, v_lop, rec.basic_salary)
        RETURNING payroll_id INTO v_payroll_id;

        FOR comp IN SELECT component_id, component_type
                      FROM salary_component
                     WHERE is_active
                     ORDER BY component_id
        LOOP
            v_amount := fn_component_amount(comp.component_id, v_earned);
            INSERT INTO payroll_detail (payroll_id, component_id, amount)
            VALUES (v_payroll_id, comp.component_id, v_amount);

            IF comp.component_type = 'Allowance' THEN
                v_allow := v_allow + v_amount;
            ELSE
                v_deduct := v_deduct + v_amount;
            END IF;
        END LOOP;

        UPDATE payroll
           SET total_allowances = v_allow,
               total_deductions = v_deduct,
               gross_salary     = v_earned + v_allow,
               net_salary       = GREATEST(v_earned + v_allow - v_deduct, 0)
         WHERE payroll_id = v_payroll_id;

        p_generated := p_generated + 1;
    END LOOP;
    CLOSE cur_emp;

    RAISE NOTICE 'Payroll % / %: % employee(s) processed', p_month, p_year, p_generated;
END;
$$;
```

How the cursor is used: `cur_emp` is declared over the active employees. Each FETCH returns one employee; the loop computes LOP days, earned basic and every active component with the three functions, writes one PAYROLL row and its PAYROLL_DETAIL lines, and skips employees already processed, so a second run creates nothing.

p_generated								
19								
emp_id	**employee**	**working_days**	**lop_days**	**basic_salary**	**gross_salary**	**total_deductions**	**net_salary**	**status**
1	Sachin Tendulkar	22	0.5	150000.00	240327.27	25620.46	214706.81	Generated
6	LeBron James	22	0.0	65000.00	108150.00	11750.00	96400.00	Generated
7	Cristiano Ronaldo	22	2.5	49500.00	73927.50	8158.75	65768.75	Generated
13	Virat Kohli	22	0.5	30000.00	50345.45	5684.09	44661.36	Generated
20	Coco Gauff	22	0.0	49500.00	83040.00	9115.00	73925.00	Generated

8.3 Trigger – trg_leave_approved

```
-- ---------------------------------------------------------------------
-- TRIGGER 1: trg_leave_approved   (AFTER UPDATE ON leave_request)
--   When a leave request becomes 'Approved', every weekday in the leave
--   range is written to ATTENDANCE with status 'Leave'. Existing rows for
--   those days are overwritten (UPSERT on the (emp_id, att_date) key).
-- ---------------------------------------------------------------------
CREATE OR REPLACE FUNCTION trgfn_leave_approved()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.status = 'Approved' AND OLD.status IS DISTINCT FROM 'Approved' THEN
        INSERT INTO attendance (emp_id, att_date, check_in, check_out, status)
        SELECT NEW.emp_id, d::DATE, NULL, NULL, 'Leave'
          FROM generate_series(NEW.start_date, NEW.end_date, INTERVAL '1 day') AS d
         WHERE EXTRACT(ISODOW FROM d) < 6
        ON CONFLICT (emp_id, att_date)
        DO UPDATE SET status = 'Leave', check_in = NULL, check_out = NULL;
    END IF;
    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS trg_leave_approved ON leave_request;
CREATE TRIGGER trg_leave_approved
AFTER UPDATE OF status ON leave_request
FOR EACH ROW
EXECUTE FUNCTION trgfn_leave_approved();
```

Approving Coco Gauff's sick leave for 15–16 October writes the leave days into ATTENDANCE automatically:

att_date	status
2026-10-15	Leave
2026-10-16	Leave

8.4 Calling Procedures from the Application

The web application talks to the database through Supabase's REST interface (PostgREST), which can call SQL **functions** over HTTP but not procedures. Thin SECURITY DEFINER wrapper functions therefore check the caller's role and then CALL the procedure. Row Level Security policies decide which rows each role may read or change; they are defined in `sql/06_supabase_auth_rls.sql`.

```
CREATE OR REPLACE FUNCTION public.api_generate_payroll(p_month INT, p_year INT)
RETURNS INT
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public
AS $$
DECLARE v_count INT := 0;
BEGIN
    IF NOT app_is_staff() THEN
        RAISE EXCEPTION 'Only Admin or Payroll staff can generate payroll';
```

```
    END IF;
    CALL generate_monthly_payroll(p_month, p_year, v_count);
    RETURN v_count;
END;
$$;

-- Row Level Security example: employees see only their own payslips
CREATE POLICY pay_read ON payroll FOR SELECT TO authenticated
    USING (app_is_staff() OR emp_id = app_emp_id());
```

9. Real-World Transaction Demonstrations

9.1 Application and Database Connectivity

The application is a React single-page app. It connects to the PostgreSQL database with the supabase-js client: SELECT/INSERT/UPDATE/DELETE requests are sent to PostgREST, procedures are reached through the api_* functions (RPC), and every request carries the signed-in user's token so Row Level Security applies. Every screen has a "DB activity" panel that lists, for each user action, the SQL or PL/pgSQL component executed, the tables involved and the result – the flow the guidelines ask for:

User operation → SQL / PL/pgSQL → Database processing → Database update / retrieval → Result

Row Level Security was verified by signing in as each role and counting the rows the database returned for the same queries:

Signed in as	employee	payroll	leave_request	attendance	Generate payroll?
Not signed in	denied	denied	denied	denied	denied
Admin (Daniel Dumile)	20	39	13	971	allowed
Payroll (Jeffery Williams)	20	39	13	971	allowed
Manager (Aubrey Graham)	8 (self, manager, team)	2 (own)	6 (own + team)	321	blocked
Employee (LeBron James)	2 (self + manager)	2 (own)	2 (own)	50 (own)	blocked

9.2 Integrated Workflow – One Monthly Payroll Cycle

The database components are not independent programs: the output of one operation is the input of the next. Attendance and leave recorded during the month feed fn_lop_days, which feeds the payroll procedure; approving leave writes attendance through a trigger; paying salaries locks the payroll through another trigger; and the reports read what the procedures produced. The table follows one month from start to finish.

Step	Actor	User operation	SQL / PL/pgSQL executed	Tables written
1	Admin	Onboards a new employee	INSERT employee – CHECK, UNIQUE and FK constraints; trg_validate_manager	employee
2	Employee	Checks in and out every day	INSERT / UPDATE attendance (RLS: own row, today)	attendance
3	Employee	Applies for leave	fn_leave_balance; INSERT leave_request (status Pending)	leave_request
4	Manager	Approves the leave	process_leave → fn_leave_balance → trg_leave_approved	leave_request, attendance
5	Payroll	Generates the month's payroll	generate_monthly_payroll – cursor cur_emp; fn_working_days, fn_lop_days, fn_component_amount	payroll, payroll_detail
6	Payroll	Pays salaries	pay_monthly_payroll – cursor cur_pay FOR UPDATE … WHERE CURRENT OF	payroll
7	Payroll	Tries to change a paid payslip	UPDATE / DELETE payroll → trg_lock_paid_payroll rejects it	– (rolled back)

Step	Actor	User operation	SQL / PL/pgSQL executed	Tables written
8	Employee	Opens their payslip	SELECT payroll ⋈ payroll_detail ⋈ salary_component (RLS: own rows)	– (read)
9	Admin / Payroll	Reviews reports	Views q04, q06, q08, q13 and the other query views	– (read)
10	Admin	Records a resignation	UPDATE employee status → trg_employee_exit → trg_validate_manager	employee, department, leave_request

Data flow: attendance (step 2) and approved unpaid leave (step 4) → fn_lop_days → loss of pay in the payroll (step 5) → payslip and reports (steps 8–9). Approved leave reaches attendance only through trg_leave_approved, so the payroll always sees the approved leave without the application having to update two tables.

Which database component runs in which transaction

Component	T1	T2	T3	T4	T5	T6	T7	T8	T9
fn_working_days				✓					
fn_component_amount				✓					
fn_lop_days				✓					
fn_leave_balance		✓	✓						✓
generate_monthly_payroll + cursor cur_emp				✓					
process_leave			✓						
pay_monthly_payroll + cursor cur_pay						✓			
trg_leave_approved			✓						
trg_lock_paid_payroll						✓	✓		
trg_validate_manager								✓	
trg_employee_exit								✓	
Constraints (PK/FK/UNIQUE/CHECK)	✓	✓	✓	✓		✓		✓	
Row Level Security	✓	✓	✓	✓	✓	✓	✓	✓	✓
Query views (Q1–Q19)									✓

T1 check-in · T2 apply leave · T3 approve leave · T4 generate payroll · T5 payslip · T6 pay salaries · T7 delete paid payroll · T8 resignation · T9 report queries. In T6 the lock trigger runs while the rows are updated from Generated to Paid and lets the change through.

9.3 Transactions

T1. Employee checks in

Question	Answer
Operation	LeBron James clicks “Check in” on his dashboard.
Tables involved	ATTENDANCE
SQL component	INSERT INTO attendance (emp_id, att_date, check_in, status) VALUES (6, CURRENT_DATE, <time>, 'Present'); – allowed by an RLS policy only for the employee’s own row and only for today.

Question	Answer
Effect on database	One attendance row is inserted. UNIQUE (emp_id, att_date) rejects a second check-in on the same day.
Result	The dashboard shows “Present · In 16:02” and a Check out button (which performs an UPDATE of check_out).

T2. Employee applies for leave

Question	Answer
Operation	LeBron applies for 2 days of Casual leave (27–28 October).
Tables involved	LEAVE_REQUEST (read: LEAVE_REQUEST, DESIGNATION for the balance)
SQL component	SELECT fn_leave_balance(6, 2026) to show the balance; INSERT INTO leave_request … with status DEFAULT 'Pending'.
Effect on database	A Pending request is inserted. CHECK (end_date ≥ start_date) and the RLS policy (own row, status Pending, no approver) are enforced.
Result	The request appears under “My requests” with status Pending.

T3. Manager approves leave

Question	Answer
Operation	Aubrey Graham approves Coco Gauff’s sick leave (request #10).
Tables involved	LEAVE_REQUEST, EMPLOYEE, DESIGNATION, ATTENDANCE
SQL / PL/pgSQL component	SELECT api_process_leave(10, 'Approved') → CALL process_leave(10, 4, 'Approved') → fn_leave_balance → TRIGGER trg_leave_approved.
Effect on database	leave_request #10 becomes Approved with approved_by = 4; the trigger inserts ‘Leave’ attendance rows for 15 and 16 October.
Result	The request leaves the approval queue; Coco’s attendance shows the two leave days (section 8.4).

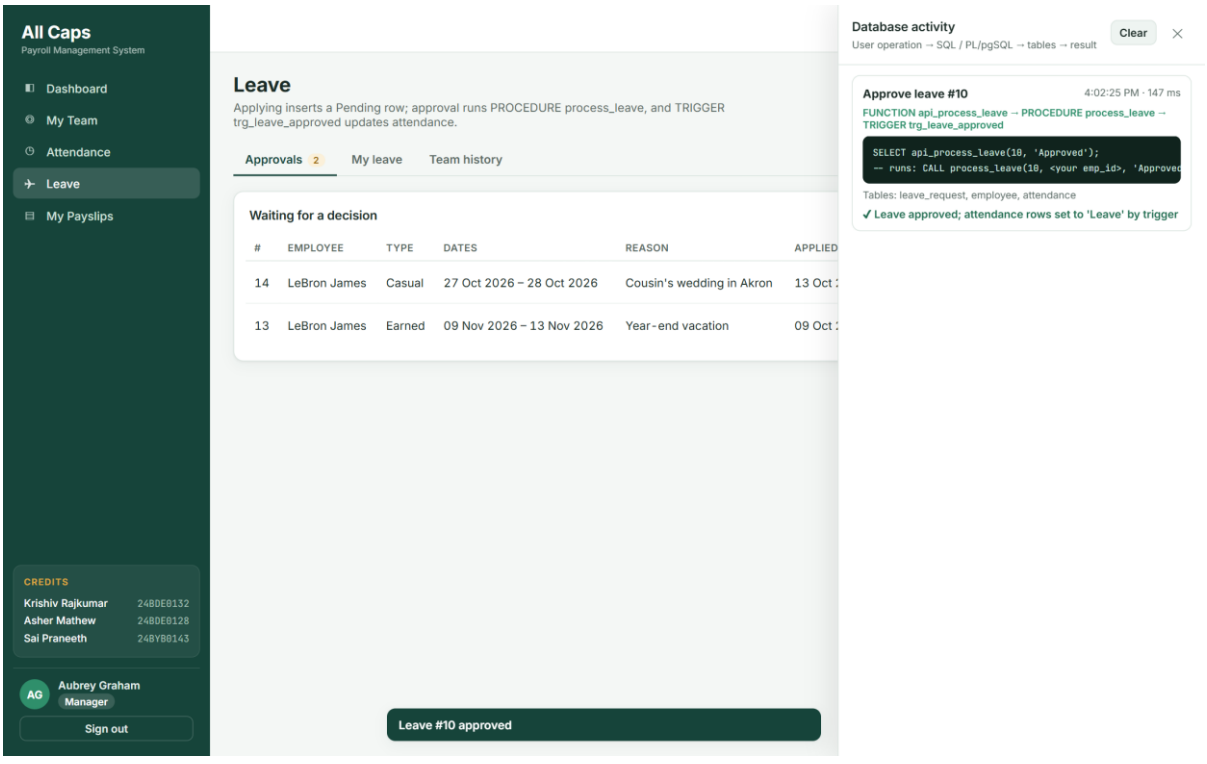


Figure 3 – After approval: the procedure and trigger chain shown in the activity panel.

T4. Payroll officer generates the monthly payroll

Question	Answer
Operation	Jeffery Williams selects October 2026 and clicks “Generate payroll”.
Tables involved	EMPLOYEE, DESIGNATION, ATTENDANCE, LEAVE_REQUEST, SALARY_COMPONENT (read); PAYROLL, PAYROLL_DETAIL (write)
SQL / PL/pgSQL component	SELECT api_generate_payroll(10, 2026) → CALL generate_monthly_payroll(10, 2026) – cursor over active employees using fn_working_days, fn_lop_days and fn_component_amount.
Effect on database	19 PAYROLL rows (status Generated) and one PAYROLL_DETAIL row per active salary component for each are inserted. Running it again inserts nothing.
Result	The payroll table lists all 19 active employees with LOP days, gross, deductions and net pay; totals are shown above it.

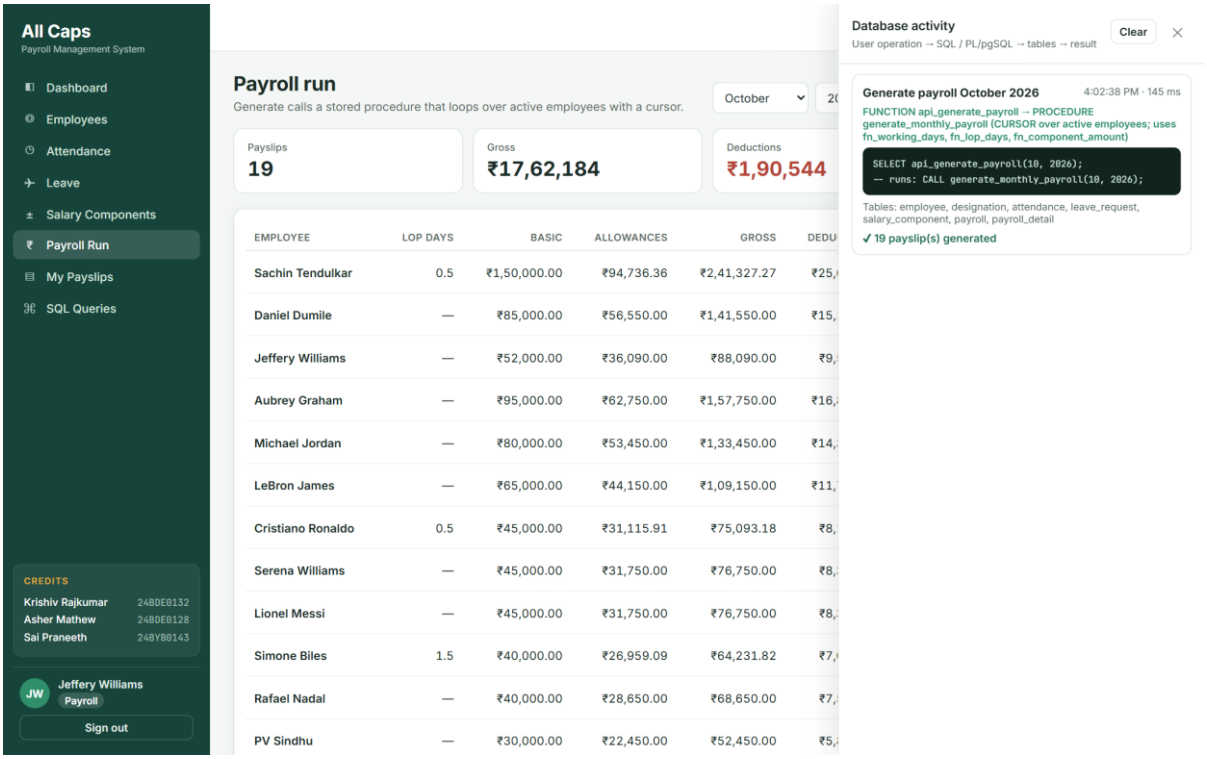


Figure 4 – Generating October payroll: one click runs the cursor-based procedure for 19 employees.

T5. Viewing an itemised payslip

Question	Answer
Operation	The payroll officer (or the employee, from My Payslips) opens a payslip.
Tables involved	PAYROLL, PAYROLL_DETAIL, SALARY_COMPONENT, EMPLOYEE, DEPARTMENT, DESIGNATION
SQL component	SELECT … FROM payroll_detail JOIN salary_component … WHERE payroll_id = ? (the M:N bridge table) plus the employee details.
Effect on database	Read only. RLS lets an employee read only their own payslips.
Result	Earnings (basic, loss of pay, allowances), deductions and net pay, ready to print.

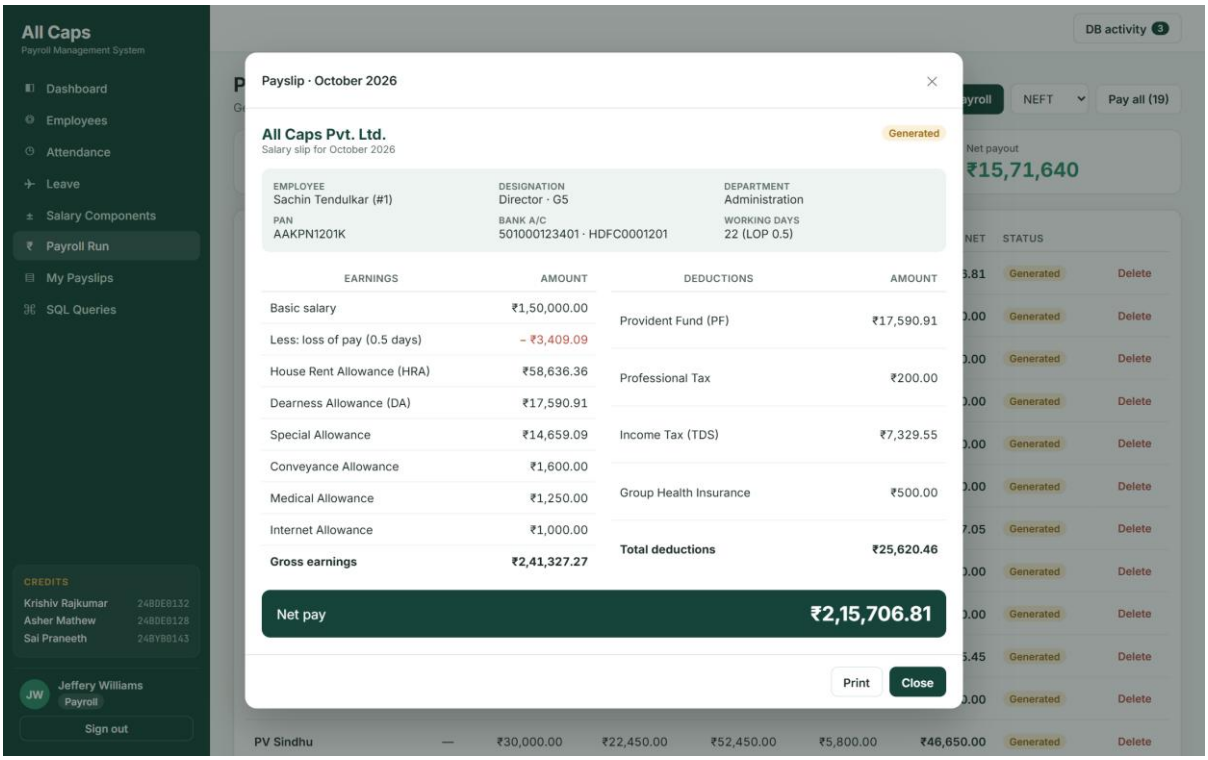


Figure 5 – Itemised payslip built from PAYROLL_DETAIL.

T6. Paying the month's salaries

Question	Answer
Operation	The payroll officer chooses NEFT and clicks "Pay all".
Tables involved	PAYROLL
SQL / PL/pgSQL component	SELECT api_pay_payroll(10, 2026, 'NEFT') → CALL pay_monthly_payroll(…) – cursor FOR UPDATE with UPDATE … WHERE CURRENT OF.
Effect on database	Every Generated row of the month becomes Paid with payment_date, payment_mode and a unique transaction_ref (CHECK: Paid ⇒ date and mode present).
Result	"19 payroll row(s) marked Paid"; the rows are now locked by trg_lock_paid_payroll.

T7. Attempt to delete a paid payslip

Question	Answer
Operation	The payroll officer clicks Delete on Sachin Tendulkar's paid October payroll.
Tables involved	PAYROLL (PAYROLL_DETAIL would cascade)
SQL / PL/pgSQL component	DELETE FROM payroll WHERE payroll_id = 40; → BEFORE DELETE trigger trg_lock_paid_payroll.
Effect on database	The trigger raises an exception, so the DELETE is rolled back and nothing changes.
Result	The error message from the database is shown to the user.

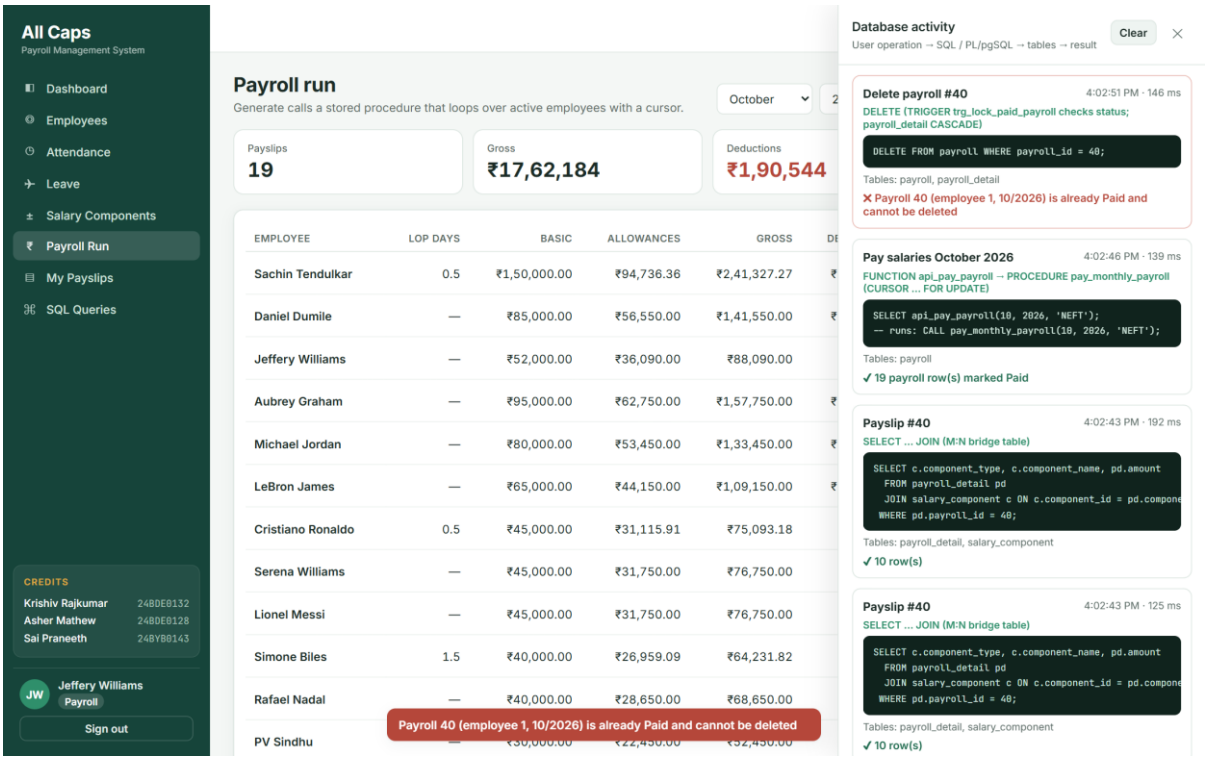


Figure 6 – The trigger blocks deleting a paid payroll; the error comes from the database.

T8. Employee resignation

Question	Answer
Operation	The Admin edits Michael Jordan's record and sets status to Resigned.
Tables involved	EMPLOYEE, DEPARTMENT, LEAVE_REQUEST
SQL / PL/pgSQL component	UPDATE employee SET status = 'Resigned' WHERE emp_id = 5; → AFTER UPDATE trigger trg_employee_exit (which in turn fires trg_validate_manager for the moved employees).
Effect on database	His three reports now report to Sachin Tendulkar, the Sales department has no head, and any pending leave of his is rejected. He is excluded from later payroll runs.
Result	Shown with before/after outputs in section 8.4 (Trigger 4).

T9. Running the report queries from the application

Question	Answer
Operation	The Admin opens SQL Queries and runs Q4 (department-wise salary cost).
Tables involved	PAYROLL, EMPLOYEE, DEPARTMENT (through view q04_department_salary_cost)
SQL component	SELECT * FROM q04_department_salary_cost; – a view created WITH (security_invoker = true) so RLS still applies.
Effect on database	Read only.
Result	The SQL text, the concepts used and the live result table are shown on one page.

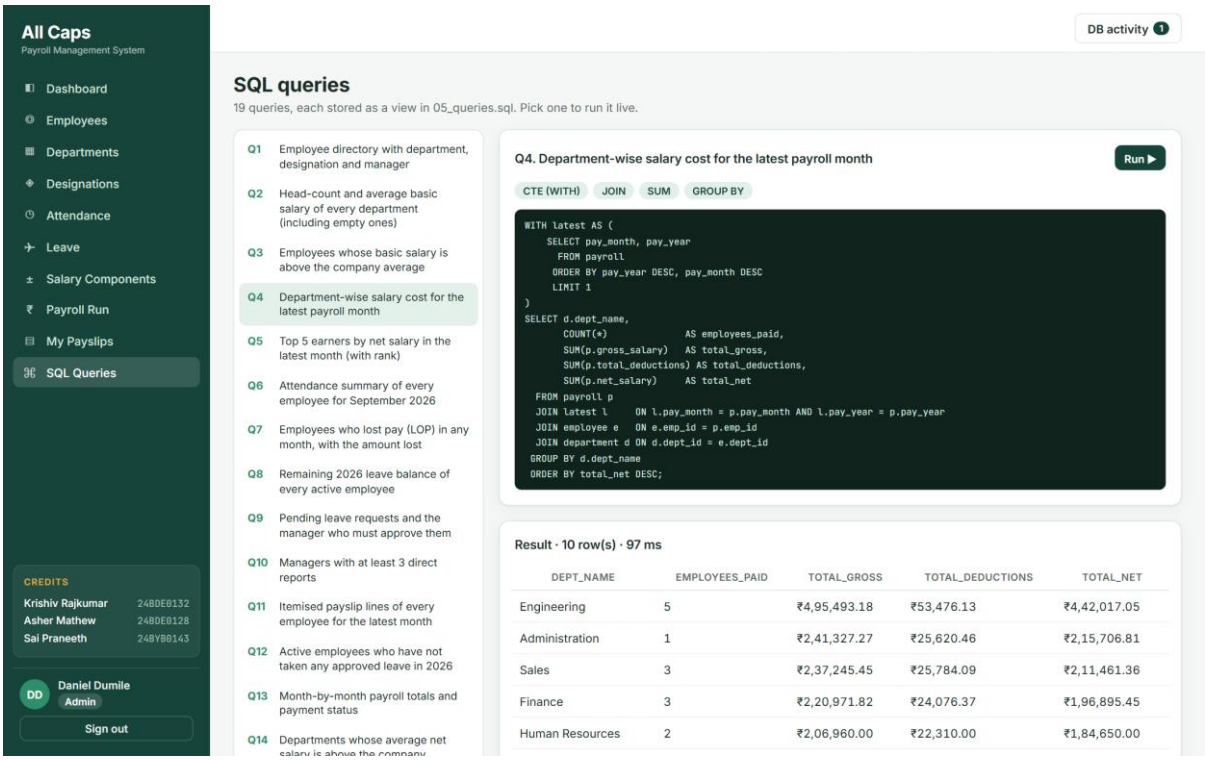


Figure 7 – The SQL Queries page running Q4 live against the database.

9.4 Reports Generated from the Database

The application turns the stored data into the reports a payroll team needs. Each report below is produced by the database (a view or a procedure) and shown in the application; the figures are actual outputs on the sample data.

Report	Produced by	Where it appears
Itemised payslip	PAYROLL + PAYROLL_DETAIL + SALARY_COMPONENT (written by generate_monthly_payroll)	Payroll Run and My Payslips pages
Monthly payroll totals and payment status	View q13_monthly_payroll_totals	Admin / Payroll dashboard
Department-wise salary cost	View q04_department_salary_cost	Dashboard bar chart and SQL Queries page
Attendance summary	View q06_attendance_summary	SQL Queries page
Leave balance of every employee	View q08_leave_balance → fn_leave_balance	SQL Queries page; each employee's Leave page
Loss-of-pay report	View q07_loss_of_pay	SQL Queries page

Monthly payroll totals (q13)

pay_year	pay_month	payslips	total_gross	total_net	payment_status
2026	8	20	1825944.25	1628413.03	All paid
2026	9	19	1696647.81	1512980.76	19 unpaid

Department-wise salary cost, September 2026 (q04)

dept_name	employees_paid	total_gross	total_deductions	total_net
Engineering	5	473034.11	51119.32	421914.79
Administration	1	235804.55	25040.91	210763.64

dept_name	employees_paid	total_gross	total_deductions	total_net
Sales	3	222518.21	24238.64	198279.57
Finance	3	218026.37	23767.28	194259.09
Human Resources	2	193374.11	20884.32	172489.79
Legal	1	114672.73	12329.54	102343.19
Quality Assurance	1	75093.18	8176.13	66917.05
Marketing	1	71890.00	7840.00	64050.00
Operations	1	49210.00	5460.00	43750.00
Customer Support	1	43024.55	4810.91	38213.64

9.5 Validation – Invalid Operations Rejected by the Database

Every business rule was tested by deliberately attempting an invalid operation. In each case the database itself refused the change, so the rule holds no matter which client sends the request.

#	Attempted operation	Rule / component	Database response	Result
1	Insert employee with PAN 'BADPAN'	CHECK constraint	new row for relation "employee" violates check constraint "employee_pan_no_check"	Pass
2	Second attendance row for the same day	UNIQUE (emp_id, att_date)	duplicate key value violates unique constraint "uq_att_emp_day"	Pass
3	Employee in a non-existent department	FOREIGN KEY	insert or update on table "employee" violates foreign key constraint "employee_dept_id_fkey"	Pass
4	Delete an employee who has payroll history	FK ON DELETE RESTRICT	update or delete on table "employee" violates foreign key constraint "payroll_emp_id_fkey" on table "payroll"	Pass
5	Manager approves leave of someone outside their team	process_leave	Employee 5 is not the manager of employee 7	Pass
6	Employee approves their own leave	process_leave	An employee cannot approve their own leave	Pass
7	Approve the same request twice	process_leave	Leave request 10 is already Approved	Pass
8	Approve 20 days of paid leave with 13 left	process_leave + fn_leave_balance	Insufficient leave balance: requested 20, available 13	Pass
9	Generate October payroll a second time	generate_monthly_payroll (skip logic)	0 new payslips created	Pass
10	Change net salary of a paid payslip	trg_lock_paid_payroll	Payroll 45 (employee 6, 10/2026) is already Paid and cannot be modified	Pass
11	Delete a paid payslip	trg_lock_paid_payroll	Payroll 45 (employee 6, 10/2026) is already Paid and cannot be deleted	Pass
12	Assign a resigned employee as manager	trg_validate_manager	Manager 19 is not an active employee	Pass
13	Create a two-person reporting cycle	trg_validate_manager	Circular reporting: employee 6 already reports to 4	Pass
14	Employee calls the payroll generation API	api_generate_payroll role check	Only Admin or Payroll staff can generate payroll	Pass
15	Read employee data without signing in	Privileges + Row Level Security	permission denied for table employee	Pass

10. Conclusion

10.1 Design Decisions and Justification

Decision	Justification
PostgreSQL (Supabase) with PL/pgSQL	A hosted database the web application can reach from anywhere; PL/pgSQL offers the same procedures, functions, cursors and triggers as Oracle PL/SQL.
Surrogate SERIAL keys, natural keys kept UNIQUE	Short, stable keys for joins and foreign keys, while e-mail, phone, PAN and bank account still cannot be duplicated.
Gross and net salary stored in PAYROLL	A deliberate snapshot: a payslip must not change when salary rules change later. Derived values that never become history (age, hours worked, leave days) are computed instead.
PAYROLL_DETAIL bridge table	Resolves the M:N relationship between payroll and salary components and keeps an auditable, itemised payslip.
Business rules inside the database	Constraints and triggers apply to every client – the web app, the SQL editor or the API – so a rule cannot be skipped by a missing UI check.
Explicit cursor in generate_monthly_payroll	Each employee needs several dependent steps (LOP, earned basic, every component, one header and many detail rows), with already-processed employees skipped; a cursor processes them one at a time in a defined order.
Cursor FOR UPDATE in pay_monthly_payroll	Locks the month's unpaid rows so two payroll officers cannot pay the same salary twice; WHERE CURRENT OF updates exactly the fetched row.
FK actions: RESTRICT / CASCADE / SET NULL	Payroll history can never be deleted with an employee (RESTRICT); attendance and leave have no meaning without the employee (CASCADE); a departing manager or approver leaves the records intact (SET NULL).
Triggers for side effects	Writing leave into attendance and cleaning up after a resignation happen in the same transaction as the change that causes them, so they cannot be forgotten or half-done.
Row Level Security + api_* wrapper functions	Each role sees only permitted rows; PostgREST cannot call procedures, so SECURITY DEFINER functions check the caller's role and then CALL the procedure.
Views with security_invoker	Report queries are written once, reused by the dashboard and the SQL Queries page, and still respect each user's row permissions.

10.2 Conclusion and Future Work

The Employee Payroll Management System has been implemented as a PostgreSQL database on Supabase with a role-based web application. The database has 8 normalised tables with complete primary key, foreign key, NOT NULL, UNIQUE, CHECK and DEFAULT constraints, populated with realistic data for a 20-employee company over three months.

Nineteen SQL queries cover joins, self-joins, aggregation, grouping with HAVING, correlated and scalar subqueries, set operations, window functions and date functions. Four functions, three procedures (two using explicit cursors) and four triggers implement the real business operations – loss-of-pay calculation, leave approval, payroll generation, salary payment and employee exit – and the database itself rejects invalid operations such as editing a paid payslip, approving one's own leave or exceeding the leave balance.

Every component has a clear purpose in the application: each user action in the web app maps to a specific SQL statement, procedure or trigger, and the DB activity panel shows that flow live during the demonstration. Possible extensions are tax-regime based TDS calculation, salary revisions with effective dates, and an audit log table populated by triggers.